8.2.2 크로스체인 자산 전송 구현하기

개요

이 고급 튜토리얼에서는 XCM(Cross-Consensus Messaging)을 사용하여 파라체인 간 자산 전송 시스템을 구현합니다. 실제 크로스체인 DeFi 애플리케이션에서 사용되는 패턴들을 학습하고, 안전하고 효율적인 자산 전송 메커니즘을 구축합니다.

학습 목표

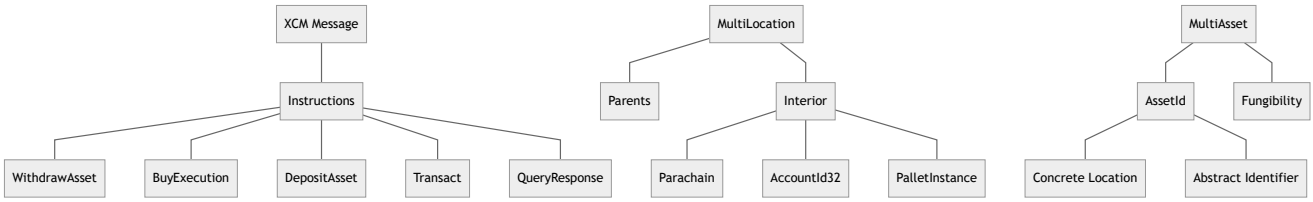
- XCM 메시지 구조와 실행 모델 이해
- 크로스체인 자산 전송 구현
- XCM 설정 및 라우팅 구성
- 멀티로케이션과 자산 식별 시스템
- 오류 처리 및 실패 복구 메커니즘

사전 요구사항

- 파라체인 템플릿 개발 경험
- XCM 기본 개념 이해
- 멀티체인 환경 구성 경험
- Rust 고급 문법 및 매크로 사용법

1단계: XCM 아키텍처 이해

1.1 XCM 메시지 구조



1.2 XCM 실행 모델

```
// XCM 메시지 예제
use xcm::latest::{
    MultiLocation, MultiAsset, MultiAssets, Instruction, Xcm,
    Junction::{Parachain, AccountId32, PalletInstance},
    Fungibility::Fungible,
    AssetId::Concrete,
    NetworkId::Any,
};

// 기본 자산 전송 메시지
let transfer_message = Xcm(vec![
    // 1. 자산 인출
    WithdrawAsset(MultiAssets::from(vec![MultiAsset {
        id: Concrete(MultiLocation::here()),
        fun: Fungible(1_000_000),
    }])),

    // 2. 실행 비용 구매
    BuyExecution {
        fees: MultiAsset {
            id: Concrete(MultiLocation::here()),
            fun: Fungible(100_000),
        },
        weight_limit: Limited(Weight::from_parts(1_000_000_000, 64 * 1024)),
    },

    // 3. 자산 예치
    DepositAsset {
        assets: Wild(AllCounted(1)),
        beneficiary: MultiLocation {
```

```
            parents: 0,
            interior: X1(AccountId32 {
                network: None,
                id: [0u8; 32], // 대상 계정
            }),
        },
    },
]);
```

2단계: 크로스체인 자산 전송 팔렛 구현

2.1 프로젝트 설정

```
# 새 팔렛 디렉토리 생성
mkdir -p pallets/xcm-asset-transfer/src
cd pallets/xcm-asset-transfer
```

2.2 Cargo.toml 설정

```
[package]
name = "pallet-xcm-asset-transfer"
version = "4.0.0-dev"
description = "XCM Asset Transfer Pallet"
authors = ["Your Name <your.email@example.com>"]
homepage = "https://substrate.io"
edition = "2021"
license = "MIT-0"
publish = false

[dependencies]
codec = { package = "parity-scale-codec", version = "3.6.1", default-features =
scale-info = { version = "2.5.0", default-features = false, features = ["derive"
serde = { version = "1.0.188", default-features = false, features = ["derive", "

# Substrate dependencies
frame-benchmarking = { version = "4.0.0-dev", default-features = false, optional
frame-support = { version = "4.0.0-dev", default-features = false, git = "https:
frame-system = { version = "4.0.0-dev", default-features = false, git = "https:/
sp-runtime = { version = "24.0.0", default-features = false, git = "https://gith
sp-std = { version = "8.0.0", default-features = false, git = "https://github.co
```

```
# XCM dependencies
xcm = { package = "staging-xcm", version = "1.0.0", default-features = false, gi
xcm-executor = { package = "staging-xcm-executor", version = "1.0.0", default-fe
xcm-builder = { package = "staging-xcm-builder", version = "1.0.0", default-feat

# Cumulus dependencies
cumulus-pallet-xcm = { version = "0.1.0", default-features = false, git = "https
cumulus-primitives-core = { version = "0.1.0", default-features = false, git = "

[dev-dependencies]
sp-core = { version = "21.0.0", git = "https://github.com/paritytech/polkadot-sd
sp-io = { version = "23.0.0", git = "https://github.com/paritytech/polkadot-sdk.
pallet-balances = { version = "4.0.0-dev", git = "https://github.com/paritytech/
pallet-assets = { version = "4.0.0-dev", git = "https://github.com/paritytech/po

[features]
default = ["std"]
std = [
    "codec/std",
    "frame-benchmarking?/std",
    "frame-support/std",
    "frame-system/std",
    "scale-info/std",
    "serde/std",
    "sp-runtime/std",
    "sp-std/std",
    "xcm/std",
    "xcm-executor/std",
    "xcm-builder/std",
    "cumulus-pallet-xcm/std",
    "cumulus-primitives-core/std",
]
runtime-benchmarks = ["frame-benchmarking/runtime-benchmarks"]
try-runtime = ["frame-support/try-runtime"]
```

2.3 기본 팔렛 구조

```rust
// pallets/xcm-asset-transfer/src/lib.rs
#![cfg_attr(not(feature = "std"), no_std)]

pub use pallet::*;

use frame_support::{
```

```
    dispatch::{DispatchResult, DispatchError},
    pallet_prelude::*,
    traits::{
        fungibles::{Inspect, Mutate, Transfer},
        tokens::Preservation,
        Get,
    },
};
use frame_system::pallet_prelude::*;
use sp_runtime::{
    traits::{Saturating, Zero, Convert},
    ArithmeticError,
};
use sp_std::{vec::Vec, boxed::Box};

use xcm::latest::{
    MultiLocation, MultiAsset, MultiAssets, Instruction, Xcm, WeightLimit,
    Junction::{Parachain, AccountId32, PalletInstance},
    Fungibility::Fungible,
    AssetId::Concrete,
    WildMultiAsset::{All, AllCounted},
    MaybeErrorCode,
};
use xcm_executor::traits::ConvertLocation;

#[frame_support::pallet]
pub mod pallet {
    use super::*;

    /// 팔렛 설정 트레이트
    #[pallet::config]
    pub trait Config: frame_system::Config {
        /// 런타임 이벤트 타입
        type RuntimeEvent: From<Event<Self>> + IsType<<Self as frame_system::Con

        /// 자산 타입
        type AssetId: Parameter + Copy + Ord + Zero;

        /// 잔액 타입
        type Balance: Parameter + Copy + Ord + Zero + Saturating + From<u32>;

        /// 자산 관리 트레이트
        type Assets: Inspect<Self::AccountId, AssetId = Self::AssetId, Balance =
            + Mutate<Self::AccountId>
            + Transfer<Self::AccountId>;
```

```rust
    /// XCM 실행기
    type XcmExecutor: ExecuteXcm<Self::RuntimeCall>;

    /// XCM 전송기
    type XcmSender: SendXcm;

    /// 계정 ID를 MultiLocation으로 변환
    type AccountIdToMultiLocation: Convert<Self::AccountId, MultiLocation>;

    /// MultiLocation을 계정 ID로 변환
    type MultiLocationToAccountId: ConvertLocation<Self::AccountId>;

    /// 현재 파라체인 ID
    #[pallet::constant]
    type SelfParaId: Get<cumulus_primitives_core::ParaId>;

    /// 릴레이 체인 토큰 위치
    #[pallet::constant]
    type RelayChainAssetLocation: Get<MultiLocation>;

    /// 기본 XCM 가중치
    #[pallet::constant]
    type DefaultXcmWeight: Get<Weight>;
}

#[pallet::pallet]
pub struct Pallet<T>(_);

/// 전송 요청 정보
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub struct TransferRequest<AccountId, AssetId, Balance> {
    pub sender: AccountId,
    pub recipient: MultiLocation,
    pub asset_id: AssetId,
    pub amount: Balance,
    pub destination: MultiLocation,
    pub fee_amount: Balance,
}

/// 전송 상태
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub enum TransferStatus {
    Pending,
    InProgress,
    Completed,
    Failed,
```

```rust
}

/// 전송 기록 저장
#[pallet::storage]
#[pallet::getter(fn transfer_records)]
pub type TransferRecords<T: Config> = StorageMap<
    _,
    Blake2_128Concat,
    u64, // Transfer ID
    TransferRequest<T::AccountId, T::AssetId, T::Balance>,
    OptionQuery,
>;

/// 전송 상태 저장
#[pallet::storage]
#[pallet::getter(fn transfer_status)]
pub type TransferStatuses<T: Config> = StorageMap<
    _,
    Blake2_128Concat,
    u64, // Transfer ID
    TransferStatus,
    ValueQuery,
>;

/// 다음 전송 ID
#[pallet::storage]
#[pallet::getter(fn next_transfer_id)]
pub type NextTransferId<T: Config> = StorageValue<_, u64, ValueQuery>;

/// 지원되는 자산 목록
#[pallet::storage]
#[pallet::getter(fn supported_assets)]
pub type SupportedAssets<T: Config> = StorageMap<
    _,
    Blake2_128Concat,
    T::AssetId,
    MultiLocation, // 자산의 원본 위치
    OptionQuery,
>;

/// 파라체인별 자산 매핑
#[pallet::storage]
#[pallet::getter(fn parachain_assets)]
pub type ParachainAssets<T: Config> = StorageDoubleMap<
    _,
    Blake2_128Concat,
```

```rust
    u32, // Parachain ID
    Blake2_128Concat,
    T::AssetId, // Local Asset ID
    MultiLocation, // Remote Asset Location
    OptionQuery,
>;

/// 팔렛 이벤트
#[pallet::event]
#[pallet::generate_deposit(pub(super) fn deposit_event)]
pub enum Event<T: Config> {
    /// 자산 전송이 시작됨 [transfer_id, sender, destination, asset_id, amount
    TransferInitiated {
        transfer_id: u64,
        sender: T::AccountId,
        destination: MultiLocation,
        asset_id: T::AssetId,
        amount: T::Balance,
    },
    /// 자산 전송이 완료됨 [transfer_id]
    TransferCompleted {
        transfer_id: u64,
    },
    /// 자산 전송이 실패함 [transfer_id, error]
    TransferFailed {
        transfer_id: u64,
        error: Vec<u8>,
    },
    /// 자산이 수신됨 [sender, recipient, asset_id, amount]
    AssetReceived {
        sender: MultiLocation,
        recipient: T::AccountId,
        asset_id: T::AssetId,
        amount: T::Balance,
    },
    /// 지원 자산이 추가됨 [asset_id, location]
    SupportedAssetAdded {
        asset_id: T::AssetId,
        location: MultiLocation,
    },
    /// XCM 메시지가 전송됨 [destination, message_hash]
    XcmMessageSent {
        destination: MultiLocation,
        message_hash: [u8; 32],
    },
}
```

```rust
/// 팔렛 오류
#[pallet::error]
pub enum Error<T> {
    /// 지원되지 않는 자산
    UnsupportedAsset,
    /// 잔액 부족
    InsufficientBalance,
    /// 잘못된 대상
    InvalidDestination,
    /// 잘못된 수신자
    InvalidRecipient,
    /// XCM 실행 실패
    XcmExecutionFailed,
    /// XCM 전송 실패
    XcmSendFailed,
    /// 전송 기록을 찾을 수 없음
    TransferNotFound,
    /// 권한 없음
    Unauthorized,
    /// 수수료 부족
    InsufficientFee,
    /// 잘못된 자산 위치
    InvalidAssetLocation,
    /// 변환 실패
    ConversionFailed,
}

/// 팔렛 디스패처블 함수들
#[pallet::call]
impl<T: Config> Pallet<T> {
    /// 다른 파라체인으로 자산 전송
    #[pallet::call_index(0)]
    #[pallet::weight(10_000)]
    pub fn transfer_to_parachain(
        origin: OriginFor<T>,
        para_id: u32,
        recipient: Vec<u8>, // AccountId32 bytes
        asset_id: T::AssetId,
        amount: T::Balance,
        fee_amount: T::Balance,
    ) -> DispatchResult {
        let sender = ensure_signed(origin)?;

        // 자산 지원 여부 확인
        let asset_location = Self::supported_assets(&asset_id)
```

```
        .ok_or(Error::<T>::UnsupportedAsset)?;

    // 잔액 확인
    let sender_balance = T::Assets::balance(asset_id, &sender);
    let total_amount = amount.saturating_add(fee_amount);
    ensure!(sender_balance >= total_amount, Error::<T>::InsufficientBala

    // 수신자 MultiLocation 생성
    let recipient_location = MultiLocation {
        parents: 1,
        interior: X2(
            Parachain(para_id),
            AccountId32 {
                network: None,
                id: Self::vec_to_account_id32(&recipient)?,
            }
        ),
    };

    // 대상 파라체인 MultiLocation 생성
    let destination = MultiLocation {
        parents: 1,
        interior: X1(Parachain(para_id)),
    };

    // 전송 ID 생성
    let transfer_id = Self::next_transfer_id();
    NextTransferId::<T>::put(transfer_id + 1);

    // 전송 기록 저장
    let transfer_request = TransferRequest {
        sender: sender.clone(),
        recipient: recipient_location.clone(),
        asset_id,
        amount,
        destination: destination.clone(),
        fee_amount,
    };

    TransferRecords::<T>::insert(&transfer_id, &transfer_request);
    TransferStatuses::<T>::insert(&transfer_id, TransferStatus::Pending)

    // 자산 잠금 (팔렛으로 전송)
    T::Assets::transfer(
        asset_id,
        &sender,
```

```
        &Self::account_id(),
        total_amount,
        Preservation::Expendable,
    )?;

    // XCM 메시지 생성 및 전송
    Self::send_xcm_transfer(
        destination,
        recipient_location,
        asset_location,
        amount,
        fee_amount,
        transfer_id,
    )?;

    // 상태 업데이트
    TransferStatuses::<T>::insert(&transfer_id, TransferStatus::InProgre

    // 이벤트 발생
    Self::deposit_event(Event::TransferInitiated {
        transfer_id,
        sender,
        destination,
        asset_id,
        amount,
    });

    Ok(())
}

/// 릴레이 체인으로 자산 전송
#[pallet::call_index(1)]
#[pallet::weight(10_000)]
pub fn transfer_to_relay_chain(
    origin: OriginFor<T>,
    recipient: Vec<u8>, // AccountId32 bytes
    amount: T::Balance,
    fee_amount: T::Balance,
) -> DispatchResult {
    let sender = ensure_signed(origin)?;

    // 릴레이 체인 토큰 확인 (일반적으로 AssetId::zero())
    let relay_asset_id = T::AssetId::zero();
    let sender_balance = T::Assets::balance(relay_asset_id, &sender);
    let total_amount = amount.saturating_add(fee_amount);
    ensure!(sender_balance >= total_amount, Error::<T>::InsufficientBala
```

```
// 수신자 MultiLocation 생성 (릴레이 체인)
let recipient_location = MultiLocation {
    parents: 1,
    interior: X1(AccountId32 {
        network: None,
        id: Self::vec_to_account_id32(&recipient)?,
    }),
};

// 릴레이 체인 대상
let destination = MultiLocation::parent();

// 전송 ID 생성
let transfer_id = Self::next_transfer_id();
NextTransferId::<T>::put(transfer_id + 1);

// 자산 잠금
T::Assets::transfer(
    relay_asset_id,
    &sender,
    &Self::account_id(),
    total_amount,
    Preservation::Expendable,
)?;

// XCM 메시지 전송
Self::send_xcm_transfer(
    destination,
    recipient_location,
    T::RelayChainAssetLocation::get(),
    amount,
    fee_amount,
    transfer_id,
)?;

// 이벤트 발생
Self::deposit_event(Event::TransferInitiated {
    transfer_id,
    sender,
    destination,
    asset_id: relay_asset_id,
    amount,
});

Ok(())
```

```
}

/// 지원 자산 추가 (관리자 전용)
#[pallet::call_index(2)]
#[pallet::weight(10_000)]
pub fn add_supported_asset(
    origin: OriginFor<T>,
    asset_id: T::AssetId,
    asset_location: MultiLocation,
) -> DispatchResult {
    ensure_root(origin)?;

    SupportedAssets::<T>::insert(&asset_id, &asset_location);

    Self::deposit_event(Event::SupportedAssetAdded {
        asset_id,
        location: asset_location,
    });

    Ok(())
}

/// 파라체인 자산 매핑 추가
#[pallet::call_index(3)]
#[pallet::weight(10_000)]
pub fn add_parachain_asset_mapping(
    origin: OriginFor<T>,
    para_id: u32,
    local_asset_id: T::AssetId,
    remote_asset_location: MultiLocation,
) -> DispatchResult {
    ensure_root(origin)?;

    ParachainAssets::<T>::insert(&para_id, &local_asset_id, &remote_asse

    Ok(())
}

/// 실패한 전송 복구
#[pallet::call_index(4)]
#[pallet::weight(10_000)]
pub fn recover_failed_transfer(
    origin: OriginFor<T>,
    transfer_id: u64,
) -> DispatchResult {
    let who = ensure_signed(origin)?;
```

```
        let transfer_request = Self::transfer_records(&transfer_id)
            .ok_or(Error::<T>::TransferNotFound)?;

        // 전송자만 복구 가능
        ensure!(transfer_request.sender == who, Error::<T>::Unauthorized);

        // 실패 상태인지 확인
        let status = Self::transfer_status(&transfer_id);
        ensure!(status == TransferStatus::Failed, Error::<T>::Unauthorized);

        // 자산 반환
        let total_amount = transfer_request.amount.saturating_add(transfer_r
        T::Assets::transfer(
            transfer_request.asset_id,
            &Self::account_id(),
            &transfer_request.sender,
            total_amount,
            Preservation::Expendable,
        )?;

        // 기록 정리
        TransferRecords::<T>::remove(&transfer_id);
        TransferStatuses::<T>::remove(&transfer_id);

        Ok(())
    }
}

/// 헬퍼 함수들
impl<T: Config> Pallet<T> {
    /// 팔렛 계정 ID
    pub fn account_id() -> T::AccountId {
        frame_support::PalletId(*b"xcm/xfer").into_account_truncating()
    }

    /// XCM 전송 메시지 생성 및 전송
    fn send_xcm_transfer(
        destination: MultiLocation,
        beneficiary: MultiLocation,
        asset_location: MultiLocation,
        amount: T::Balance,
        fee_amount: T::Balance,
        transfer_id: u64,
    ) -> DispatchResult {
        // 전송할 자산 정의
```

```
let asset = MultiAsset {
    id: Concrete(asset_location),
    fun: Fungible(amount.into()),
};

// 수수료 자산 정의
let fee_asset = MultiAsset {
    id: Concrete(asset_location),
    fun: Fungible(fee_amount.into()),
};

// XCM 메시지 구성
let xcm_message = Xcm(vec![
    // 1. 자산 인출
    WithdrawAsset(MultiAssets::from(vec![
        asset.clone(),
        fee_asset.clone(),
    ])),

    // 2. 실행 비용 구매
    BuyExecution {
        fees: fee_asset,
        weight_limit: WeightLimit::Limited(T::DefaultXcmWeight::get(
    },

    // 3. 자산 예치
    DepositAsset {
        assets: Wild(AllCounted(1)),
        beneficiary,
    },

    // 4. 응답 보고 (선택사항)
    ReportError(QueryResponseInfo {
        destination: MultiLocation {
            parents: 0,
            interior: X1(Parachain(T::SelfParaId::get().into())),
        },
        query_id: transfer_id,
        max_weight: T::DefaultXcmWeight::get(),
    }),
]);

// XCM 메시지 전송
let message_hash = xcm_message.using_encoded(sp_io::hashing::blake2_

T::XcmSender::send_xcm(destination, xcm_message)
```

```
        .map_err(|_| Error::<T>::XcmSendFailed)?;

    // 이벤트 발생
    Self::deposit_event(Event::XcmMessageSent {
        destination,
        message_hash,
    });

    Ok(())
}

/// Vec<u8>를 [u8; 32]로 변환
fn vec_to_account_id32(vec: &[u8]) -> Result<[u8; 32], Error<T>> {
    if vec.len() != 32 {
        return Err(Error::<T>::ConversionFailed);
    }

    let mut array = [0u8; 32];
    array.copy_from_slice(vec);
    Ok(array)
}

/// 전송 상태 업데이트
pub fn update_transfer_status(transfer_id: u64, status: TransferStatus)
    TransferStatuses::<T>::insert(&transfer_id, status);
}

/// 자산 수신 처리
pub fn handle_asset_received(
    sender: MultiLocation,
    recipient: T::AccountId,
    asset_id: T::AssetId,
    amount: T::Balance,
) -> DispatchResult {
    // 자산 발행 또는 전송
    T::Assets::mint_into(asset_id, &recipient, amount)?;

    // 이벤트 발생
    Self::deposit_event(Event::AssetReceived {
        sender,
        recipient,
        asset_id,
        amount,
    });

    Ok(())
```

```rust
}

/// 멀티홉 전송 경로 계산
pub fn calculate_transfer_route(
    source_para: u32,
    dest_para: u32,
) -> Vec<MultiLocation> {
    if source_para == dest_para {
        return vec![];
    }

    // 직접 연결 확인
    if Self::has_direct_connection(source_para, dest_para) {
        return vec![
            MultiLocation {
                parents: 1,
                interior: X1(Parachain(dest_para)),
            }
        ];
    }

    // 릴레이 체인을 통한 라우팅
    vec![
        MultiLocation::parent(), // 릴레이 체인
        MultiLocation {
            parents: 1,
            interior: X1(Parachain(dest_para)),
        },
    ]
}

/// 직접 연결 확인 (실제 구현에서는 더 복잡한 로직)
fn has_direct_connection(source_para: u32, dest_para: u32) -> bool {
    // 간단한 예제: 인접한 파라체인 ID는 직접 연결
    (source_para as i32 - dest_para as i32).abs() == 1
}

/// 전송 수수료 계산
pub fn calculate_transfer_fee(
    destination: &MultiLocation,
    asset_id: T::AssetId,
    amount: T::Balance,
) -> T::Balance {
    // 기본 수수료 계산 로직
    let base_fee = T::Balance::from(1000u32);
    let amount_fee = amount / T::Balance::from(1000u32); // 0.1%
```

```
            // 거리에 따른 추가 수수료
            let distance_multiplier = match destination.interior {
                X1(Parachain(_)) => T::Balance::from(1u32),
                _ => T::Balance::from(2u32), // 릴레이 체인 또는 더 복잡한 경로
            };

            base_fee.saturating_add(amount_fee).saturating_mul(distance_multipli
        }
    }

    /// XCM 메시지 처리를 위한 훅
    impl<T: Config> Pallet<T> {
        /// XCM 실행 결과 처리
        pub fn handle_xcm_result(
            transfer_id: u64,
            result: Result<(), xcm::latest::Error>,
        ) {
            match result {
                Ok(()) => {
                    Self::update_transfer_status(transfer_id, TransferStatus::Co
                    Self::deposit_event(Event::TransferCompleted { transfer_id }
                },
                Err(error) => {
                    Self::update_transfer_status(transfer_id, TransferStatus::Fa
                    let error_bytes = format!("{:?}", error).into_bytes();
                    Self::deposit_event(Event::TransferFailed {
                        transfer_id,
                        error: error_bytes,
                    });
                },
            }
        }
    }
}

// XCM 설정을 위한 추가 타입들
use xcm::latest::{
    WithdrawAsset, BuyExecution, DepositAsset, ReportError, QueryResponseInfo,
    WildMultiAsset::Wild,
    Instruction::{WithdrawAsset, BuyExecution, DepositAsset, ReportError},
    Junction::X1,
    Junctions::X2,
};
```

이제 XCM 자산 전송 팔렛의 기본 구조를 완성했습니다. 계속해서 나머지 부분을 작성하겠습니다.

<function_calls>

append## 3단계: XCM 설정 및 런타임 통합

3.1 런타임 XCM 설정

```
// runtime/src/xcm_config.rs
use super::{
    AccountId, AllPalletsWithSystem, Assets, Balance, Balances, ParachainInfo, P
    PolkadotXcm, Runtime, RuntimeCall, RuntimeEvent, RuntimeOrigin, WeightToFee,
};
use frame_support::{
    match_types, parameter_types,
    traits::{ConstU32, Contains, Everything, Nothing},
    weights::Weight,
};
use xcm::latest::{prelude::*, Weight as XCMWeight};
use xcm_builder::{
    AccountId32Aliases, AllowExplicitUnpaidExecutionFrom, AllowTopLevelPaidExecu
    CurrencyAdapter, EnsureXcmOrigin, FixedWeightBounds, IsConcrete, NativeAsset
    RelayChainAsNative, SiblingParachainAsNative, SiblingParachainConvertsVia,
    SignedAccountId32AsNative, SignedToAccountId32, SovereignSignedViaLocation,
    UsingComponents, WithComputedOrigin,
};
use xcm_executor::{XcmExecutor, Config as XcmConfig};

parameter_types! {
    pub const RelayLocation: MultiLocation = MultiLocation::parent();
    pub const RelayNetwork: Option<NetworkId> = None;
    pub RelayChainOrigin: RuntimeOrigin = cumulus_pallet_xcm::Origin::Relay.into
    pub UniversalLocation: InteriorMultiLocation = Parachain(ParachainInfo::para
}

/// XCM 오리진을 로컬 오리진으로 변환
pub type XcmOriginToTransactDispatchOrigin = (
    // 주권 계정 (Sovereign account)
    SovereignSignedViaLocation<LocationToAccountId, RuntimeOrigin>,
    // 릴레이 체인 오리진
    RelayChainAsNative<RelayChainOrigin, RuntimeOrigin>,
    // 형제 파라체인 오리진
    SiblingParachainAsNative<cumulus_pallet_xcm::Origin, RuntimeOrigin>,
    // 서명된 계정 오리진
```

```
    SignedAccountId32AsNative<RelayNetwork, RuntimeOrigin>,
    // XCM 오리진을 계정으로 변환
    XcmPassthrough<RuntimeOrigin>,
);

parameter_types! {
    pub const UnitWeightCost: XCMWeight = XCMWeight::from_parts(1_000_000_000, 6
    pub const MaxInstructions: u32 = 100;
    pub const MaxAssetsIntoHolding: u32 = 64;
}

match_types! {
    pub type ParentOrParentsExecutivePlurality: impl Contains<MultiLocation> = {
        MultiLocation { parents: 1, interior: Here } |
        MultiLocation { parents: 1, interior: X1(Plurality { id: BodyId::Executi
    };
}

/// 자산 변환기
pub type LocationToAccountId = (
    // 부모 체인 (릴레이 체인)
    ParentIsPreset<AccountId>,
    // 형제 파라체인
    SiblingParachainConvertsVia<Sibling, AccountId>,
    // AccountId32 계정
    AccountId32Aliases<RelayNetwork, AccountId>,
);

/// 로컬 자산 변환기
pub type LocalAssetTransactor = CurrencyAdapter<
    // 네이티브 통화 사용
    Balances,
    // 네이티브 자산 매처
    IsConcrete<RelayLocation>,
    // 위치를 계정으로 변환
    LocationToAccountId,
    // 계정 ID 타입
    AccountId,
    // 체크 계정 (사용하지 않음)
    (),
>;

/// XCM 실행 허용 조건
pub type XcmRouter = WithComputedOrigin<
    (
        // 부모 또는 부모의 실행 복수체에서 무료 실행 허용
```

```
        AllowExplicitUnpaidExecutionFrom<ParentOrParentsExecutivePlurality>,
        // 최상위 유료 실행 허용
        AllowTopLevelPaidExecutionFrom<Everything>,
        // 가중치 크레딧 사용
        TakeWeightCredit,
    ),
    UniversalLocation,
    ConstU32<8>,
>;

/// 장벽 (Barrier) - XCM 실행 조건
pub type Barrier = TakeWeightCredit;

/// 가중치 거래자
pub type Trader = UsingComponents<WeightToFee, RelayLocation, AccountId, Balance

/// XCM 설정 구조체
pub struct XcmConfig;
impl xcm_executor::Config for XcmConfig {
    type RuntimeCall = RuntimeCall;
    type XcmSender = XcmRouter;
    type AssetTransactor = LocalAssetTransactor;
    type OriginConverter = XcmOriginToTransactDispatchOrigin;
    type IsReserve = NativeAsset;
    type IsTeleporter = (); // 텔레포트 비활성화
    type UniversalLocation = UniversalLocation;
    type Barrier = Barrier;
    type Weigher = FixedWeightBounds<UnitWeightCost, RuntimeCall, MaxInstruction
    type Trader = Trader;
    type ResponseHandler = PolkadotXcm;
    type AssetTrap = PolkadotXcm;
    type AssetClaims = PolkadotXcm;
    type SubscriptionService = PolkadotXcm;
    type PalletInstancesInfo = AllPalletsWithSystem;
    type MaxAssetsIntoHolding = MaxAssetsIntoHolding;
    type AssetLocker = ();
    type AssetExchanger = ();
    type FeeManager = ();
    type MessageExporter = ();
    type UniversalAliases = Nothing;
    type CallDispatcher = RuntimeCall;
    type SafeCallFilter = Everything;
    type Aliasers = Nothing;
}

/// XCM 실행기 타입
```

```rust
pub type LocalXcmExecutor = XcmExecutor<XcmConfig>;

/// XCM 패스스루 오리진
#[derive(Clone, Debug, PartialEq, Eq, Encode, Decode)]
pub struct XcmPassthrough<RuntimeOrigin>(PhantomData<RuntimeOrigin>);
impl<RuntimeOrigin> TryFrom<MultiLocation> for XcmPassthrough<RuntimeOrigin> {
    type Error = MultiLocation;
    fn try_from(o: MultiLocation) -> Result<Self, Self::Error> {
        Ok(XcmPassthrough(PhantomData))
    }
}
impl<RuntimeOrigin> Into<RuntimeOrigin> for XcmPassthrough<RuntimeOrigin>
where
    RuntimeOrigin: From<pallet_xcm::Origin>,
{
    fn into(self) -> RuntimeOrigin {
        pallet_xcm::Origin::Xcm(MultiLocation::here()).into()
    }
}
```

3.2 런타임 팔렛 설정

```rust
// runtime/src/lib.rs에 추가

use xcm_config::{LocalXcmExecutor, XcmConfig, XcmOriginToTransactDispatchOrigin,

// XCM Asset Transfer 팔렛 설정
parameter_types! {
    pub const DefaultXcmWeight: Weight = Weight::from_parts(1_000_000_000, 64 *
}

impl pallet_xcm_asset_transfer::Config for Runtime {
    type RuntimeEvent = RuntimeEvent;
    type AssetId = u32;
    type Balance = Balance;
    type Assets = Assets;
    type XcmExecutor = LocalXcmExecutor;
    type XcmSender = XcmRouter;
    type AccountIdToMultiLocation = xcm_config::AccountIdToMultiLocation;
    type MultiLocationToAccountId = xcm_config::LocationToAccountId;
    type SelfParaId = parachain_info::Pallet<Runtime>;
    type RelayChainAssetLocation = xcm_config::RelayLocation;
    type DefaultXcmWeight = DefaultXcmWeight;
```

```
}

// PolkadotXcm 팔렛 설정
impl pallet_xcm::Config for Runtime {
    type RuntimeEvent = RuntimeEvent;
    type SendXcmOrigin = EnsureXcmOrigin<RuntimeOrigin, XcmOriginToTransactDispa
    type XcmRouter = XcmRouter;
    type ExecuteXcmOrigin = EnsureXcmOrigin<RuntimeOrigin, XcmOriginToTransactDi
    type XcmExecuteFilter = Nothing;
    type XcmExecutor = LocalXcmExecutor;
    type XcmTeleportFilter = Everything;
    type XcmReserveTransferFilter = Nothing;
    type Weigher = FixedWeightBounds<UnitWeightCost, RuntimeCall, MaxInstruction
    type UniversalLocation = UniversalLocation;
    type RuntimeOrigin = RuntimeOrigin;
    type RuntimeCall = RuntimeCall;
    const VERSION_DISCOVERY_QUEUE_SIZE: u32 = 100;
    type AdvertisedXcmVersion = pallet_xcm::CurrentXcmVersion;
    type Currency = Balances;
    type CurrencyMatcher = ();
    type TrustedLockers = ();
    type SovereignAccountOf = LocationToAccountId;
    type MaxLockers = ConstU32<8>;
    type WeightInfo = pallet_xcm::TestWeightInfo;
}

// construct_runtime! 매크로에 팔렛 추가
construct_runtime!(
    pub struct Runtime {
        // ... 기존 팔렛들
        XcmAssetTransfer: pallet_xcm_asset_transfer,
        PolkadotXcm: pallet_xcm,
        CumulusXcm: cumulus_pallet_xcm,
        XcmpQueue: cumulus_pallet_xcmp_queue,
    }
);
```

4단계: 고급 XCM 기능 구현

4.1 멀티홉 전송

```rust
// pallets/xcm-asset-transfer/src/lib.rs에 추가

impl<T: Config> Pallet<T> {
    /// 멀티홉 자산 전송
    #[pallet::call_index(5)]
    #[pallet::weight(10_000)]
    pub fn multi_hop_transfer(
        origin: OriginFor<T>,
        route: Vec<MultiLocation>,
        final_recipient: Vec<u8>,
        asset_id: T::AssetId,
        amount: T::Balance,
        max_fee: T::Balance,
    ) -> DispatchResult {
        let sender = ensure_signed(origin)?;

        // 경로 유효성 검사
        ensure!(!route.is_empty(), Error::<T>::InvalidDestination);
        ensure!(route.len() <= 5, Error::<T>::InvalidDestination); // 최대 5홉

        // 자산 지원 여부 확인
        let asset_location = Self::supported_assets(&asset_id)
            .ok_or(Error::<T>::UnsupportedAsset)?;

        // 잔액 확인
        let sender_balance = T::Assets::balance(asset_id, &sender);
        let total_amount = amount.saturating_add(max_fee);
        ensure!(sender_balance >= total_amount, Error::<T>::InsufficientBalance)

        // 전송 ID 생성
        let transfer_id = Self::next_transfer_id();
        NextTransferId::<T>::put(transfer_id + 1);

        // 자산 잠금
        T::Assets::transfer(
            asset_id,
            &sender,
            &Self::account_id(),
            total_amount,
            Preservation::Expendable,
        )?;

        // 멀티홉 XCM 메시지 생성
        Self::send_multi_hop_xcm(
            route,
```

```
        final_recipient,
        asset_location,
        amount,
        max_fee,
        transfer_id,
    )?;

    Ok(())
}

/// 멀티홉 XCM 메시지 전송
fn send_multi_hop_xcm(
    route: Vec<MultiLocation>,
    final_recipient: Vec<u8>,
    asset_location: MultiLocation,
    amount: T::Balance,
    max_fee: T::Balance,
    transfer_id: u64,
) -> DispatchResult {
    let mut current_message = Self::create_final_deposit_message(
        final_recipient,
        asset_location,
        amount,
    )?;

    // 역순으로 메시지 래핑
    for (i, hop) in route.iter().enumerate().rev() {
        if i == 0 {
            // 첫 번째 홉: 자산 인출 및 전송
            current_message = Self::wrap_with_withdraw_and_forward(
                current_message,
                asset_location,
                amount,
                max_fee / (route.len() as u32).into(),
                hop.clone(),
            )?;
        } else {
            // 중간 홉: 전달만
            current_message = Self::wrap_with_forward(
                current_message,
                hop.clone(),
            )?;
        }
    }

    // 첫 번째 대상으로 메시지 전송
```

```
    T::XcmSender::send_xcm(route[0], current_message)
        .map_err(|_| Error::<T>::XcmSendFailed)?;

    Ok(())
}

/// 최종 예치 메시지 생성
fn create_final_deposit_message(
    recipient: Vec<u8>,
    asset_location: MultiLocation,
    amount: T::Balance,
) -> Result<Xcm<()>, Error<T>> {
    let recipient_location = MultiLocation {
        parents: 0,
        interior: X1(AccountId32 {
            network: None,
            id: Self::vec_to_account_id32(&recipient)?,
        }),
    };

    Ok(Xcm(vec![
        DepositAsset {
            assets: Wild(AllCounted(1)),
            beneficiary: recipient_location,
        },
    ]))
}

/// 인출 및 전달 메시지로 래핑
fn wrap_with_withdraw_and_forward(
    inner_message: Xcm<()>,
    asset_location: MultiLocation,
    amount: T::Balance,
    fee_amount: T::Balance,
    next_hop: MultiLocation,
) -> Result<Xcm<()>, Error<T>> {
    let asset = MultiAsset {
        id: Concrete(asset_location),
        fun: Fungible(amount.into()),
    };

    let fee_asset = MultiAsset {
        id: Concrete(asset_location),
        fun: Fungible(fee_amount.into()),
    };
```

```
        Ok(Xcm(vec![
            WithdrawAsset(MultiAssets::from(vec![asset, fee_asset.clone()])),
            BuyExecution {
                fees: fee_asset,
                weight_limit: WeightLimit::Limited(T::DefaultXcmWeight::get()),
            },
            InitiateReserveWithdraw {
                assets: Wild(AllCounted(1)),
                reserve: next_hop,
                xcm: inner_message,
            },
        ]))
    }

    /// 전달 메시지로 래핑
    fn wrap_with_forward(
        inner_message: Xcm<()>,
        next_hop: MultiLocation,
    ) -> Result<Xcm<()>, Error<T>> {
        Ok(Xcm(vec![
            InitiateReserveWithdraw {
                assets: Wild(AllCounted(1)),
                reserve: next_hop,
                xcm: inner_message,
            },
        ]))
    }
}
```

4.2 자산 스왑과 전송 결합

```
// 크로스체인 스왑 및 전송
impl<T: Config> Pallet<T> {
    /// 스왑 후 전송
    #[pallet::call_index(6)]
    #[pallet::weight(10_000)]
    pub fn swap_and_transfer(
        origin: OriginFor<T>,
        input_asset_id: T::AssetId,
        output_asset_id: T::AssetId,
        input_amount: T::Balance,
        min_output_amount: T::Balance,
        destination: MultiLocation,
        recipient: Vec<u8>,
```

```
        fee_amount: T::Balance,
    ) -> DispatchResult {
        let sender = ensure_signed(origin)?;

        // 입력 자산 잔액 확인
        let sender_balance = T::Assets::balance(input_asset_id, &sender);
        ensure!(sender_balance >= input_amount, Error::<T>::InsufficientBalance)

        // 자산 위치 확인
        let input_asset_location = Self::supported_assets(&input_asset_id)
            .ok_or(Error::<T>::UnsupportedAsset)?;
        let output_asset_location = Self::supported_assets(&output_asset_id)
            .ok_or(Error::<T>::UnsupportedAsset)?;

        // 자산 잠금
        T::Assets::transfer(
            input_asset_id,
            &sender,
            &Self::account_id(),
            input_amount,
            Preservation::Expendable,
        )?;

        // 스왑 및 전송 XCM 메시지 생성
        let recipient_location = MultiLocation {
            parents: 1,
            interior: X2(
                Parachain(Self::extract_para_id(&destination)?),
                AccountId32 {
                    network: None,
                    id: Self::vec_to_account_id32(&recipient)?,
                }
            ),
        };

        let xcm_message = Xcm(vec![
            // 1. 입력 자산 인출
            WithdrawAsset(MultiAssets::from(vec![MultiAsset {
                id: Concrete(input_asset_location),
                fun: Fungible(input_amount.into()),
            }])),

            // 2. DEX에서 스왑 실행
            Transact {
                origin_kind: OriginKind::SovereignAccount,
                require_weight_at_most: T::DefaultXcmWeight::get(),
```

```
            call: Self::create_swap_call(
                input_asset_id,
                output_asset_id,
                input_amount,
                min_output_amount,
            )?.encode().into(),
        },

        // 3. 수수료 구매
        BuyExecution {
            fees: MultiAsset {
                id: Concrete(output_asset_location),
                fun: Fungible(fee_amount.into()),
            },
            weight_limit: WeightLimit::Limited(T::DefaultXcmWeight::get()),
        },

        // 4. 결과 자산 전송
        InitiateReserveWithdraw {
            assets: Wild(AllCounted(1)),
            reserve: destination,
            xcm: Xcm(vec![
                DepositAsset {
                    assets: Wild(AllCounted(1)),
                    beneficiary: recipient_location,
                },
            ]),
        },
    ]);

    // XCM 메시지 전송
    T::XcmSender::send_xcm(destination, xcm_message)
        .map_err(|_| Error::<T>::XcmSendFailed)?;

    Ok(())
}

/// 스왑 호출 생성 (DEX 팔렛과 연동)
fn create_swap_call(
    input_asset: T::AssetId,
    output_asset: T::AssetId,
    input_amount: T::Balance,
    min_output: T::Balance,
) -> Result<RuntimeCall, Error<T>> {
    // 실제 구현에서는 DEX 팔렛의 스왑 함수 호출
    // 예: pallet_defi_dex::Call::swap_exact_tokens_for_tokens
```

```
        Err(Error::<T>::ConversionFailed) // 임시 구현
    }

    /// MultiLocation에서 파라체인 ID 추출
    fn extract_para_id(location: &MultiLocation) -> Result<u32, Error<T>> {
        match &location.interior {
            X1(Parachain(id)) => Ok(*id),
            X2(Parachain(id), _) => Ok(*id),
            _ => Err(Error::<T>::InvalidDestination),
        }
    }
}
```

5단계: 테스트 코드 작성

5.1 Mock 런타임

```
// pallets/xcm-asset-transfer/src/mock.rs
use crate as pallet_xcm_asset_transfer;
use frame_support::{
    derive_impl, parameter_types,
    traits::{AsEnsureOriginWithArg, ConstU16, ConstU32, ConstU64, ConstU128, Eve
    weights::Weight,
};
use sp_core::H256;
use sp_runtime::{
    traits::{BlakeTwo256, IdentityLookup, Convert},
    BuildStorage,
};
use xcm::latest::{MultiLocation, Junction::Parachain};
use xcm_builder::{ParentIsPreset, SiblingParachainConvertsVia, AccountId32Aliase
use xcm_executor::XcmExecutor;

type Block = frame_system::mocking::MockBlock<Test>;

frame_support::construct_runtime!(
    pub enum Test {
        System: frame_system,
        Balances: pallet_balances,
        Assets: pallet_assets,
        XcmAssetTransfer: pallet_xcm_asset_transfer,
    }
```

```
);

#[derive_impl(frame_system::config_preludes::TestDefaultConfig as frame_system::
impl frame_system::Config for Test {
    type BaseCallFilter = Everything;
    type BlockWeights = ();
    type BlockLength = ();
    type DbWeight = ();
    type RuntimeOrigin = RuntimeOrigin;
    type RuntimeCall = RuntimeCall;
    type Nonce = u64;
    type Hash = H256;
    type Hashing = BlakeTwo256;
    type AccountId = u64;
    type Lookup = IdentityLookup<Self::AccountId>;
    type Block = Block;
    type RuntimeEvent = RuntimeEvent;
    type BlockHashCount = ConstU64<250>;
    type Version = ();
    type PalletInfo = PalletInfo;
    type AccountData = pallet_balances::AccountData<u128>;
    type OnNewAccount = ();
    type OnKilledAccount = ();
    type SystemWeightInfo = ();
    type SS58Prefix = ConstU16<42>;
    type OnSetCode = ();
    type MaxConsumers = ConstU32<16>;
}

impl pallet_balances::Config for Test {
    type MaxLocks = ConstU32<50>;
    type MaxReserves = ();
    type ReserveIdentifier = [u8; 8];
    type Balance = u128;
    type RuntimeEvent = RuntimeEvent;
    type DustRemoval = ();
    type ExistentialDeposit = ConstU128<500>;
    type AccountStore = System;
    type WeightInfo = pallet_balances::weights::SubstrateWeight<Test>;
    type FreezeIdentifier = ();
    type MaxFreezes = ();
    type RuntimeHoldReason = ();
    type RuntimeFreezeReason = ();
}

impl pallet_assets::Config for Test {
```

```
    type RuntimeEvent = RuntimeEvent;
    type Balance = u128;
    type AssetId = u32;
    type AssetIdParameter = codec::Compact<u32>;
    type Currency = Balances;
    type CreateOrigin = AsEnsureOriginWithArg<frame_system::EnsureSigned<u64>>;
    type ForceOrigin = frame_system::EnsureRoot<u64>;
    type AssetDeposit = ConstU128<100>;
    type AssetAccountDeposit = ConstU128<1>;
    type MetadataDepositBase = ConstU128<10>;
    type MetadataDepositPerByte = ConstU128<1>;
    type ApprovalDeposit = ConstU128<1>;
    type StringLimit = ConstU32<50>;
    type Freezer = ();
    type Extra = ();
    type CallbackHandle = ();
    type WeightInfo = pallet_assets::weights::SubstrateWeight<Test>;
    type RemoveItemsLimit = ConstU32<1000>;
}

// Mock XCM 구성
parameter_types! {
    pub const RelayLocation: MultiLocation = MultiLocation::parent();
    pub const DefaultXcmWeight: Weight = Weight::from_parts(1_000_000_000, 64 *
    pub const SelfParaId: cumulus_primitives_core::ParaId = cumulus_primitives_c
}

pub struct MockXcmSender;
impl xcm::latest::SendXcm for MockXcmSender {
    type Ticket = ();

    fn validate(
        _destination: &mut Option<MultiLocation>,
        _message: &mut Option<xcm::latest::Xcm<()>>,
    ) -> xcm::latest::SendResult<Self::Ticket> {
        Ok((((), MultiAssets::new()))
    }

    fn deliver(_ticket: Self::Ticket) -> Result<xcm::latest::XcmHash, xcm::lates
        Ok([0; 32])
    }
}

pub struct MockXcmExecutor;
impl xcm::latest::ExecuteXcm<RuntimeCall> for MockXcmExecutor {
    type Prepared = ();
```

```
    fn prepare(
        _message: xcm::latest::Xcm<RuntimeCall>,
    ) -> Result<Self::Prepared, xcm::latest::ExecuteXcmError> {
        Ok(())
    }

    fn execute(
        _origin: impl Into<MultiLocation>,
        _pre: Self::Prepared,
        _id: &mut xcm::latest::XcmHash,
        _weight_credit: Weight,
    ) -> xcm::latest::Outcome {
        xcm::latest::Outcome::Complete(Weight::zero())
    }
}

pub struct AccountIdToMultiLocation;
impl Convert<u64, MultiLocation> for AccountIdToMultiLocation {
    fn convert(account: u64) -> MultiLocation {
        MultiLocation {
            parents: 0,
            interior: xcm::latest::Junctions::X1(xcm::latest::Junction::AccountI
                network: None,
                id: account.to_le_bytes().into(),
            }),
        }
    }
}

pub type LocationToAccountId = (
    ParentIsPreset<u64>,
    SiblingParachainConvertsVia<cumulus_primitives_core::ParaId, u64>,
    AccountId32Aliases<(), u64>,
);

impl pallet_xcm_asset_transfer::Config for Test {
    type RuntimeEvent = RuntimeEvent;
    type AssetId = u32;
    type Balance = u128;
    type Assets = Assets;
    type XcmExecutor = MockXcmExecutor;
    type XcmSender = MockXcmSender;
    type AccountIdToMultiLocation = AccountIdToMultiLocation;
    type MultiLocationToAccountId = LocationToAccountId;
    type SelfParaId = SelfParaId;
```

```
    type RelayChainAssetLocation = RelayLocation;
    type DefaultXcmWeight = DefaultXcmWeight;
}

pub fn new_test_ext() -> sp_io::TestExternalities {
    let mut t = frame_system::GenesisConfig::<Test>::default().build_storage().u

    pallet_balances::GenesisConfig::<Test> {
        balances: vec![
            (1, 1_000_000_000_000),
            (2, 1_000_000_000_000),
            (3, 1_000_000_000_000),
        ],
    }
    .assimilate_storage(&mut t)
    .unwrap();

    let mut ext = sp_io::TestExternalities::new(t);
    ext.execute_with(|| System::set_block_number(1));
    ext
}
```

5.2 단위 테스트

```
// pallets/xcm-asset-transfer/src/tests.rs
use super::*;
use crate::{mock::*, Error, Event};
use frame_support::{assert_noop, assert_ok, traits::Get};
use xcm::latest::{MultiLocation, Junction::Parachain};

#[test]
fn add_supported_asset_works() {
    new_test_ext().execute_with(|| {
        let asset_location = MultiLocation {
            parents: 1,
            interior: xcm::latest::Junctions::X1(Parachain(1000)),
        };

        assert_ok!(XcmAssetTransfer::add_supported_asset(
            RuntimeOrigin::root(),
            1,
            asset_location.clone()
        ));
```

```
        assert_eq!(XcmAssetTransfer::supported_assets(1), Some(asset_location.cl

        System::assert_last_event(Event::SupportedAssetAdded {
            asset_id: 1,
            location: asset_location,
        }.into());
    });
}

#[test]
fn transfer_to_parachain_works() {
    new_test_ext().execute_with(|| {
        // 자산 설정
        setup_assets();

        // 파라체인 전송
        assert_ok!(XcmAssetTransfer::transfer_to_parachain(
            RuntimeOrigin::signed(1),
            2000,
            vec![2u8; 32],
            1,
            100_000,
            10_000
        ));

        // 전송 기록 확인
        let transfer_record = XcmAssetTransfer::transfer_records(0).unwrap();
        assert_eq!(transfer_record.sender, 1);
        assert_eq!(transfer_record.asset_id, 1);
        assert_eq!(transfer_record.amount, 100_000);

        // 상태 확인
        assert_eq!(XcmAssetTransfer::transfer_status(0), TransferStatus::InProgr

        // 이벤트 확인
        System::assert_has_event(Event::TransferInitiated {
            transfer_id: 0,
            sender: 1,
            destination: MultiLocation {
                parents: 1,
                interior: xcm::latest::Junctions::X1(Parachain(2000)),
            },
            asset_id: 1,
            amount: 100_000,
        }.into());
```

```rust
    });
}

#[test]
fn transfer_fails_with_insufficient_balance() {
    new_test_ext().execute_with(|| {
        setup_assets();

        // 잔액보다 많은 금액 전송 시도
        assert_noop!(
            XcmAssetTransfer::transfer_to_parachain(
                RuntimeOrigin::signed(1),
                2000,
                vec![2u8; 32],
                1,
                2_000_000, // 잔액보다 큰 금액
                10_000
            ),
            Error::<Test>::InsufficientBalance
        );
    });
}

#[test]
fn transfer_fails_with_unsupported_asset() {
    new_test_ext().execute_with(|| {
        // 자산 생성만 하고 지원 자산으로 등록하지 않음
        assert_ok!(Assets::create(RuntimeOrigin::signed(1), 1.into(), 1, 1));
        assert_ok!(Assets::mint(RuntimeOrigin::signed(1), 1.into(), 1, 1_000_000

        assert_noop!(
            XcmAssetTransfer::transfer_to_parachain(
                RuntimeOrigin::signed(1),
                2000,
                vec![2u8; 32],
                1,
                100_000,
                10_000
            ),
            Error::<Test>::UnsupportedAsset
        );
    });
}

#[test]
fn recover_failed_transfer_works() {
```

```rust
    new_test_ext().execute_with(|| {
        setup_assets();

        // 전송 시작
        assert_ok!(XcmAssetTransfer::transfer_to_parachain(
            RuntimeOrigin::signed(1),
            2000,
            vec![2u8; 32],
            1,
            100_000,
            10_000
        ));

        // 실패 상태로 설정 (실제로는 XCM 실행 결과에 의해 설정됨)
        XcmAssetTransfer::update_transfer_status(0, TransferStatus::Failed);

        let initial_balance = Assets::balance(1, &1);

        // 실패한 전송 복구
        assert_ok!(XcmAssetTransfer::recover_failed_transfer(
            RuntimeOrigin::signed(1),
            0
        ));

        // 자산이 반환되었는지 확인
        let final_balance = Assets::balance(1, &1);
        assert_eq!(final_balance, initial_balance + 110_000); // amount + fee

        // 기록이 정리되었는지 확인
        assert_eq!(XcmAssetTransfer::transfer_records(0), None);
    });
}

#[test]
fn multi_hop_transfer_works() {
    new_test_ext().execute_with(|| {
        setup_assets();

        let route = vec![
            MultiLocation {
                parents: 1,
                interior: xcm::latest::Junctions::X1(Parachain(1000)),
            },
            MultiLocation {
                parents: 1,
                interior: xcm::latest::Junctions::X1(Parachain(2000)),
```

```
            },
        ];

        assert_ok!(XcmAssetTransfer::multi_hop_transfer(
            RuntimeOrigin::signed(1),
            route,
            vec![3u8; 32],
            1,
            100_000,
            20_000
        ));

        // 자산이 잠겼는지 확인
        let pallet_balance = Assets::balance(1, &XcmAssetTransfer::account_id())
        assert_eq!(pallet_balance, 120_000);
    });
}

#[test]
fn calculate_transfer_fee_works() {
    new_test_ext().execute_with(|| {
        let parachain_dest = MultiLocation {
            parents: 1,
            interior: xcm::latest::Junctions::X1(Parachain(2000)),
        };

        let relay_dest = MultiLocation::parent();

        let para_fee = XcmAssetTransfer::calculate_transfer_fee(&parachain_dest,
        let relay_fee = XcmAssetTransfer::calculate_transfer_fee(&relay_dest, 1,

        // 릴레이 체인 전송이 더 비싸야 함
        assert!(relay_fee > para_fee);
    });
}

// 헬퍼 함수
fn setup_assets() {
    // 자산 생성
    assert_ok!(Assets::create(RuntimeOrigin::signed(1), 1.into(), 1, 1));
    assert_ok!(Assets::mint(RuntimeOrigin::signed(1), 1.into(), 1, 1_000_000));

    // 지원 자산으로 등록
    let asset_location = MultiLocation {
        parents: 0,
        interior: xcm::latest::Junctions::X1(xcm::latest::Junction::GeneralIndex
```

```
    };
    assert_ok!(XcmAssetTransfer::add_supported_asset(
        RuntimeOrigin::root(),
        1,
        asset_location
    ));
}
```

6단계: 프론트엔드 통합

6.1 JavaScript API 래퍼

```
// frontend/xcm-transfer-api.js
import { ApiPromise, WsProvider } from '@polkadot/api';
import { Keyring } from '@polkadot/keyring';

class XcmTransferAPI {
    constructor(wsEndpoint = 'ws://localhost:9944') {
        this.wsEndpoint = wsEndpoint;
        this.api = null;
        this.keyring = new Keyring({ type: 'sr25519' });
    }

    async connect() {
        const wsProvider = new WsProvider(this.wsEndpoint);
        this.api = await ApiPromise.create({ provider: wsProvider });
        console.log('Connected to parachain node');
    }

    async transferToParachain(signer, paraId, recipient, assetId, amount, feeAmo
        const tx = this.api.tx.xcmAssetTransfer.transferToParachain(
            paraId,
            recipient,
            assetId,
            amount,
            feeAmount
        );

        return this.sendTransaction(tx, signer, 'TransferInitiated');
    }

    async transferToRelayChain(signer, recipient, amount, feeAmount) {
```

```
    const tx = this.api.tx.xcmAssetTransfer.transferToRelayChain(
        recipient,
        amount,
        feeAmount
    );

    return this.sendTransaction(tx, signer, 'TransferInitiated');
}

async multiHopTransfer(signer, route, finalRecipient, assetId, amount, maxFe
    const tx = this.api.tx.xcmAssetTransfer.multiHopTransfer(
        route,
        finalRecipient,
        assetId,
        amount,
        maxFee
    );

    return this.sendTransaction(tx, signer, 'TransferInitiated');
}

async getTransferRecord(transferId) {
    const record = await this.api.query.xcmAssetTransfer.transferRecords(tra
    return record.isSome ? record.unwrap().toJSON() : null;
}

async getTransferStatus(transferId) {
    const status = await this.api.query.xcmAssetTransfer.transferStatuses(tr
    return status.toString();
}

async getSupportedAssets() {
    const assets = await this.api.query.xcmAssetTransfer.supportedAssets.ent
    return assets.map(([key, value]) => ({
        assetId: key.args[0].toNumber(),
        location: value.unwrap().toJSON()
    }));
}

async calculateTransferFee(destination, assetId, amount) {
    // 체인 상태를 기반으로 수수료 계산
    // 실제 구현에서는 런타임 API 호출 또는 로컬 계산
    const baseFee = 1000;
    const amountFee = Math.floor(amount * 0.001); // 0.1%

    const isRelayChain = destination.parents === 1 &&
```

```
                    (!destination.interior || destination.interior === 'H
    const distanceMultiplier = isRelayChain ? 2 : 1;

    return (baseFee + amountFee) * distanceMultiplier;
}

async monitorTransfer(transferId, callback) {
    const unsubscribe = await this.api.query.system.events((events) => {
        events.forEach((record) => {
            const { event } = record;

            if (event.section === 'xcmAssetTransfer') {
                if (event.method === 'TransferCompleted') {
                    const [completedId] = event.data;
                    if (completedId.toNumber() === transferId) {
                        callback({ status: 'completed', transferId });
                    }
                } else if (event.method === 'TransferFailed') {
                    const [failedId, error] = event.data;
                    if (failedId.toNumber() === transferId) {
                        callback({
                            status: 'failed',
                            transferId,
                            error: error.toString()
                        });
                    }
                }
            }
        });
    });

    return unsubscribe;
}

async sendTransaction(tx, signer, eventMethod) {
    return new Promise((resolve, reject) => {
        tx.signAndSend(signer, ({ status, events }) => {
            if (status.isInBlock) {
                events.forEach(({ event }) => {
                    if (event.section === 'xcmAssetTransfer' && event.method
                        resolve(event.data.toJSON());
                    }
                });
            } else if (status.isError) {
                reject(new Error('Transaction failed'));
            }
```

```
            });
        });
    }

    // 멀티로케이션 헬퍼 함수들
    createParachainLocation(paraId) {
        return {
            parents: 1,
            interior: {
                X1: { Parachain: paraId }
            }
        };
    }

    createAccountLocation(paraId, accountId) {
        return {
            parents: 1,
            interior: {
                X2: [
                    { Parachain: paraId },
                    { AccountId32: { network: null, id: accountId } }
                ]
            }
        };
    }

    createRelayChainLocation() {
        return {
            parents: 1,
            interior: 'Here'
        };
    }
}

export default XcmTransferAPI;
```

6.2 React 컴포넌트

```
// frontend/components/XcmTransferInterface.jsx
import React, { useState, useEffect } from 'react';
import XcmTransferAPI from '../xcm-transfer-api';

const XcmTransferInterface = ({ api, account }) => {
```

```
const [xcmApi, setXcmApi] = useState(null);
const [transferType, setTransferType] = useState('parachain');
const [paraId, setParaId] = useState('');
const [recipient, setRecipient] = useState('');
const [assetId, setAssetId] = useState('');
const [amount, setAmount] = useState('');
const [estimatedFee, setEstimatedFee] = useState('0');
const [supportedAssets, setSupportedAssets] = useState([]);
const [transferHistory, setTransferHistory] = useState([]);
const [loading, setLoading] = useState(false);

useEffect(() => {
    if (api) {
        const xcmTransferApi = new XcmTransferAPI();
        xcmTransferApi.api = api;
        setXcmApi(xcmTransferApi);
        loadSupportedAssets(xcmTransferApi);
    }
}, [api]);

const loadSupportedAssets = async (xcmTransferApi) => {
    try {
        const assets = await xcmTransferApi.getSupportedAssets();
        setSupportedAssets(assets);
    } catch (error) {
        console.error('Error loading supported assets:', error);
    }
};

const calculateFee = async () => {
    if (!xcmApi || !amount || !assetId) return;

    try {
        const destination = transferType === 'parachain'
            ? xcmApi.createParachainLocation(parseInt(paraId))
            : xcmApi.createRelayChainLocation();

        const fee = await xcmApi.calculateTransferFee(
            destination,
            parseInt(assetId),
            parseInt(amount)
        );

        setEstimatedFee(fee.toString());
    } catch (error) {
        console.error('Error calculating fee:', error);
```

```
    }
};

const executeTransfer = async () => {
    if (!xcmApi || !account) return;

    setLoading(true);
    try {
        let result;
        const recipientBytes = new TextEncoder().encode(recipient).slice(0,
        const paddedRecipient = new Uint8Array(32);
        paddedRecipient.set(recipientBytes);

        if (transferType === 'parachain') {
            result = await xcmApi.transferToParachain(
                account,
                parseInt(paraId),
                Array.from(paddedRecipient),
                parseInt(assetId),
                parseInt(amount),
                parseInt(estimatedFee)
            );
        } else {
            result = await xcmApi.transferToRelayChain(
                account,
                Array.from(paddedRecipient),
                parseInt(amount),
                parseInt(estimatedFee)
            );
        }

        // 전송 모니터링 시작
        const transferId = result.transfer_id || result.transferId;
        monitorTransfer(transferId);

        alert(`Transfer initiated! Transfer ID: ${transferId}`);

        // 폼 초기화
        setAmount('');
        setRecipient('');
        setEstimatedFee('0');

    } catch (error) {
        console.error('Transfer failed:', error);
        alert('Transfer failed: ' + error.message);
    } finally {
```

```
        setLoading(false);
    }
};

const monitorTransfer = async (transferId) => {
    if (!xcmApi) return;

    const unsubscribe = await xcmApi.monitorTransfer(transferId, (update) =>
        console.log('Transfer update:', update);

        setTransferHistory(prev => {
            const existing = prev.find(t => t.transferId === transferId);
            if (existing) {
                return prev.map(t =>
                    t.transferId === transferId
                        ? { ...t, status: update.status, error: update.error
                        : t
                );
            } else {
                return [...prev, {
                    transferId,
                    status: update.status,
                    timestamp: new Date().toISOString(),
                    error: update.error
                }];
            }
        });

        if (update.status === 'completed' || update.status === 'failed') {
            unsubscribe();
        }
    });
};

useEffect(() => {
    calculateFee();
}, [amount, assetId, paraId, transferType]);

return (
    <div className="xcm-transfer-interface">
        <h2>Cross-Chain Asset Transfer</h2>

        <div className="transfer-form">
            <div className="form-group">
                <label>Transfer Type:</label>
                <select
```

```
            value={transferType}
            onChange={(e) => setTransferType(e.target.value)}
        >
            <option value="parachain">To Parachain</option>
            <option value="relay">To Relay Chain</option>
        </select>
    </div>

    {transferType === 'parachain' && (
        <div className="form-group">
            <label>Destination Parachain ID:</label>
            <input
                type="number"
                value={paraId}
                onChange={(e) => setParaId(e.target.value)}
                placeholder="Enter parachain ID"
            />
        </div>
    )}

    <div className="form-group">
        <label>Recipient Account:</label>
        <input
            type="text"
            value={recipient}
            onChange={(e) => setRecipient(e.target.value)}
            placeholder="Enter recipient account ID"
        />
    </div>

    <div className="form-group">
        <label>Asset:</label>
        <select
            value={assetId}
            onChange={(e) => setAssetId(e.target.value)}
        >
            <option value="">Select Asset</option>
            {supportedAssets.map(asset => (
                <option key={asset.assetId} value={asset.assetId}>
                    Asset ID: {asset.assetId}
                </option>
            ))}
        </select>
    </div>

    <div className="form-group">
```

```
            <label>Amount:</label>
            <input
                type="number"
                value={amount}
                onChange={(e) => setAmount(e.target.value)}
                placeholder="Enter amount"
            />
        </div>

        <div className="form-group">
            <label>Estimated Fee:</label>
            <input
                type="text"
                value={estimatedFee}
                readOnly
                className="readonly"
            />
        </div>

        <button
            onClick={executeTransfer}
            disabled={loading || !amount || !recipient || !assetId}
            className="transfer-button"
        >
            {loading ? 'Transferring...' : 'Execute Transfer'}
        </button>
    </div>

    <div className="transfer-history">
        <h3>Transfer History</h3>
        {transferHistory.length === 0 ? (
            <p>No transfers yet</p>
        ) : (
            <div className="history-list">
                {transferHistory.map(transfer => (
                    <div key={transfer.transferId} className="history-it
                        <div className="transfer-id">ID: {transfer.trans
                        <div className={`status ${transfer.status}`}>
                            Status: {transfer.status}
                        </div>
                        <div className="timestamp">
                            {new Date(transfer.timestamp).toLocaleString
                        </div>
                        {transfer.error && (
                            <div className="error">Error: {transfer.erro
                        )}
```

```
                            </div>
                        ))}
                    </div>
                )}
            </div>
        </div>
    );
};

export default XcmTransferInterface;
```

7단계: 실제 테스트 및 배포

7.1 로컬 테스트넷 구성

```
# Zombienet 설정 파일
# zombienet/xcm-test-network.toml
[settings]
timeout = 1000

[relaychain]
default_image = "parity/polkadot:latest"
default_command = "./polkadot"
chain = "rococo-local"

  [[relaychain.nodes]]
  name = "alice"
  validator = true

  [[relaychain.nodes]]
  name = "bob"
  validator = true

[[parachains]]
id = 2000
chain = "local"

  [parachains.collator]
  name = "collator-2000"
  image = "parachain-xcm:latest"
  command = "./parachain-template-node"
```

```toml
[[parachains]]
id = 2001
chain = "local"

  [parachains.collator]
  name = "collator-2001"
  image = "parachain-xcm:latest"
  command = "./parachain-template-node"
```

7.2 XCM 테스트 시나리오

```
# 1. 네트워크 시작
./zombienet spawn zombienet/xcm-test-network.toml

# 2. 자산 생성 및 설정
# Polkadot.js Apps에서:
# - Assets 팔렛으로 테스트 토큰 생성
# - XCM Asset Transfer 팔렛으로 지원 자산 등록

# 3. 크로스체인 전송 테스트
# - 파라체인 2000 → 파라체인 2001
# - 파라체인 → 릴레이 체인
# - 릴레이 체인 → 파라체인

# 4. 멀티홉 전송 테스트
# - 파라체인 2000 → 릴레이 체인 → 파라체인 2001
```

7.3 성능 및 보안 테스트

```rust
// 벤치마킹 코드
#[cfg(feature = "runtime-benchmarks")]
mod benchmarking {
    use super::*;
    use frame_benchmarking::v2::*;
    use frame_system::RawOrigin;

    #[benchmarks]
    mod benchmarks {
        use super::*;

        #[benchmark]
        fn transfer_to_parachain() {
```

```
        let caller: T::AccountId = whitelisted_caller();
        let para_id = 2000u32;
        let recipient = vec![1u8; 32];
        let asset_id = T::AssetId::zero();
        let amount = T::Balance::from(100_000u32);
        let fee_amount = T::Balance::from(10_000u32);

        // 자산 설정
        let asset_location = MultiLocation::parent();
        let _ = Pallet::<T>::add_supported_asset(
            RawOrigin::Root.into(),
            asset_id,
            asset_location,
        );

        #[extrinsic_call]
        transfer_to_parachain(
            RawOrigin::Signed(caller),
            para_id,
            recipient,
            asset_id,
            amount,
            fee_amount,
        );

        assert_eq!(Pallet::<T>::next_transfer_id(), 1);
    }

    #[benchmark]
    fn multi_hop_transfer() {
        let caller: T::AccountId = whitelisted_caller();
        let route = vec![
            MultiLocation { parents: 1, interior: X1(Parachain(1000)) },
            MultiLocation { parents: 1, interior: X1(Parachain(2000)) },
        ];
        let recipient = vec![1u8; 32];
        let asset_id = T::AssetId::zero();
        let amount = T::Balance::from(100_000u32);
        let max_fee = T::Balance::from(20_000u32);

        #[extrinsic_call]
        multi_hop_transfer(
            RawOrigin::Signed(caller),
            route,
            recipient,
            asset_id,
```

```
                amount,
                max_fee,
            );
        }

        impl_benchmark_test_suite!(Pallet, crate::mock::new_test_ext(), crate::m
    }
}
```

요약

이 고급 튜토리얼에서는 다음을 구현했습니다:

1. **완전한 XCM 자산 전송 시스템**: 파라체인 간, 릴레이 체인과의 양방향 전송
2. **고급 XCM 기능**: 멀티홉 전송, 스왑과 전송 결합, 실패 복구
3. **포괄적인 설정**: XCM 설정, 런타임 통합, 보안 고려사항
4. **실용적인 도구**: JavaScript API, React 컴포넌트, 모니터링 시스템
5. **테스트 및 검증**: 단위 테스트, 통합 테스트, 벤치마킹

다음 튜토리얼에서는 거버넌스 시스템 커스터마이징을 다루겠습니다.

참고 자료

- XCM 포맷 명세
- Cumulus XCM 가이드
- Polkadot XCM 문서
- XCM 실행기 설정